

- 大项目里的 AI：用特征测试做安全重构，用 GitHub 打通协作管线
 - 🎯 核心内容
 - 🔭 开场之前：SDD+TDD 跟 OpenSpec、Superpowers 是什么关系
 - 📖 这篇讲这些
 - 0. 🔍 大项目里 AI 的三个风险
 - 0.1 为什么 AI 把它们放大了
 - 1. 💡 思维跃迁：小项目 AI 帮你写，大项目 AI 帮你填
 - 1.1 上下文工程：不是塞得越多越好
 - 1.2 AI 当活文档用
 - 2. 🔬 让 AI 读懂遗留代码
 - 2.1 AI 应该找出哪些现状
 - 2.2 带走：架构梳理行动卡
 - 3. 🔒 把现状写成 spec，并生成特征测试
 - 3.0 重构之前先回答三个问题
 - 3.0.1 行为面清单：不要只盯返回值
 - 3.1 为什么 AI 让"不敢动"变成了"敢动"
 - 3.2 锁住现状的四步
 - 3.2.1 特征测试 vs 普通测试
 - 3.3 带走：特征测试行动卡
 - 4. 🔧 小步重构，并用红绿测试验证行为没变
 - 4.1 怎么把大改造拆成安全小步
 - 4.2 带走：安全重构执行卡
 - 5. 🧩 另一种难：问题藏在结构里
 - 5.1 带走：结构梳理行动卡
 - 6. 📁 Git 管理如何保障质量
 - 6.1 几个 Git 动作在质量控制里的实际作用
 - 6.2 Git 和特征测试怎么配合
 - 7. 📖 让 spec 活起来：从一次性快照到状态账本
 - 7.0 为什么光有 README 不够
 - 7.1 issue 分类是 spec 治理的入口
 - 7.2 带走：spec ↔ issue 治理行动卡
 - 8. ⚙️ 把 AI 产出放进 GitHub workflow
 - 8.1 workflow 强在哪："自动 + 强制"
 - 8.1.1 workflow 可以逐步升级，不要一上来就做复杂
 - 8.2 从变更链路看这条管线
 - 8.2.1 AI review 可以用，但不能当门禁
 - 8.3 带走：GitHub AI 管线行动卡

- 9.  收束：生成—验证—修正质量飞轮
 - 9.0 AI 比人更需要制度
 - 9.1 人退到哪里
-  总结
-  常见翻车处理

大项目里的 AI：用特征测试做安全重构，用 GitHub 打通协作管线

模块 7 「AI Coding 带来的范式变革」· 第 2 课

小李上个月接手了一个五年的老仓库。代码一百多万行，最早写的那拨人走了一半。他让 AI 改一个函数——函数本身改得挺好，上线就把另一个模块的结算金额算错了。AI 没看到那个模块偷偷依赖了这个函数的副作用。

大项目里用 AI，问题不在它会不会写代码。问题是它看不全、你不敢让它改、改完的产出你也管不住。

这篇讲的就是这三件事怎么解决。每一步都配了一张操作清单，拿回自己项目就能用。

核心内容

§0-§1 先想清楚问题是什么

- 大项目里 AI 的三个风险：看不全、改不敢、产出管不住——为什么换模型没用
- 人跟 AI 分工：人定边界，AI 在边界里填实现；演进式设计里，定方向是人的活
- 上下文工程：不是喂得越多越好；AI 可以当活文档用，先读老代码产 spec

§2-§5 怎么安全地重构老代码

- 让 AI 只读、先交结构理解（入口在哪、职责怎么分的、状态藏在哪、哪些地方可疑）
- 把现状写成 spec，用特征测试锁住当前行为——就算看着像 bug 也先锁
- 小步改：保留旧入口，内部一块一块拆，每步跑测试，故意改坏一处看红灯亮不亮
- 另一种难：代码已经拆成模块了，但模块之间靠全局变量和 import 副作用暗中耦合
- 过去不敢动老代码是因为补测试太贵，AI 把这个成本打下来了

§6 把 Git 当质量控制轨道用

- issue 划边界 → branch 隔离 → commit 定位 → diff 审查 → PR 变成团队对话
- 跟特征测试配合：先补安全网，再小步重构

§7 让 spec 活着，别让它烂掉

- 规则文件 + 分状态 spec 目录 + spec ↔ issue 循环
- 模板在 [code/04_live_spec_template/](#)

§8 用 GitHub workflow 当门禁

- 最小可用：lint + pytest，设成 required check，红了合不进去
- AI review 只能辅助，门禁必须是确定性的
- 可跑样例在 [code/05_github_pipeline/](#)
- 结构复杂度也有行动卡：[结构梳理行动卡](#)

§9 收成飞轮

- 特征测试 + CI 门禁 = 同一件事：生成 → 验证 → 修正
- AI 边界用例自测（不是 ML 对抗样本）
- AI 比人更需要制度；人退到定边界、定标准、按 merge
- 从超级个体到超级团队（下一篇展开）

开场之前：SDD+TDD 跟 OpenSpec、Superpowers 是什么关系

讲完 SDD+TDD 之后，有同学问：我们讲的这套方法，和 OpenSpec、Superpowers 这些工具到底什么关系？它们是不是比 Plan Mode 更强的"计划模式"？

这个问题值得先说清楚。

Plan Mode 解决的是"开工前先停一下，想清楚再动手"。但计划留在 session 里，不天然变成长期资产，也不强制测试和审查。单人临时任务够用。

OpenSpec 往前多走了一步。每次变更生成独立目录——[proposal.md](#) → [design.md](#) → [tasks.md](#) → [archive/](#)——需求从聊天碎片变成了可版本化、可审查的工程对象。它不绑定某个 coding agent，支持 20 多种。适合中大型功能、多人协作、需求经常变的项目。

Superpowers 走的是另一条路。它不是"先计划"而是"按方法论执行"——TDD 技能先写失败测试再实现，调试技能先做根因分析再修，执行计划里嵌 code review 检查点。适

合希望 AI 按工程纪律工作、而不是自由发挥的场景。

SDD+TDD 的位置： 它在做 Superpowers 和 OpenSpec 各自的一部分事——SDD 把需求变成可版本化的规格（OpenSpec 的方向），TDD 把工程纪律固化成红绿循环（Superpowers 的方向）。区别在于，它不依赖某一个外部工具，而是把这两个方法论当作人和 **AI** 之间的协作协议——不管底层用的 coding agent 是 Codex 还是 Claude Code，这套协议不变。

理解了这个，再往下看上下文工程、特征测试、GitHub 管线——就不是"又学了一堆工具"，而是把同一套协作协议从一个人扩展到整个代码库。

这篇讲这些

小节	你要做的事	带走
§0	看清楚大项目里 AI 到底哪里危险	—
§1	把脑子掰过来：你不是写代码的，你是定边界的	—
§2	让 AI 只读老代码，交出结构理解	架构梳理行动卡
§3	把现状写成 spec，用特征测试锁住	特征测试行动卡
§4	小步改，每步跑测试，故意改坏一次看红灯	安全重构执行卡
§5	另一种难：代码拆了模块但逻辑还缠在一起	结构梳理行动卡
§6	把 Git 当成质量控制轨道，不只是版本历史	—
§7	让 spec 活起来，别让它变成过期文档	spec ↔ issue 治理行动卡
§8	把 lint + pytest 设成 GitHub 门禁	GitHub AI 管线行动卡
§9	收成一个飞轮：生成 → 验证 → 修正	质量飞轮行动卡

0. 大项目里 AI 的三个风险

个人项目里，AI 帮你写一个函数、改一个页面，出不了大事。大项目不一样。

看不全。 上下文窗口就那么点大，代码库上百万行。AI 只看到巴掌大一块，没看到的地方，就是它最容易改坏的地方。它不是在乱改——在自己看到的局部里，它的选择往往是合理的。问题出在它看不到那些依赖上。

改不敢。老代码还在线上跑，没人说得清它为什么这么写。最可怕的不是 AI 改不动——是它太能改了。一通重构下来，代码漂亮了，但行为是不是和昨天一样？没人敢签字。

产出管不住。一个人用 AI，产出已经翻倍。一个团队里每人一个 agent，PR 像水一样往主干涌。以前靠“谁写的谁负责”，现在负责的人来得及看吗？主干有没有一道不看人脸色的闸门？

换更强的模型没用。窗口还是那么大，老代码还是没人说得清，PR 还是看不过来。能做到的，是每一步操作留在 Git 里、每一步改动有测试锁着、任何一步出问题能退回去。

0.1 为什么 AI 把它们放大了

人改大项目也会犯错。但人慢，一天写不了几个函数；人怕，线上事故的教训会让他手软。AI 不慢、不怕、不累。它改完一个马上改下一个，而且从不觉得自己可能错。

这种“自信高产”会把原来藏在流程里的小裂缝撑开。issue 写得含糊，人可能会问一句，AI 直接按自己理解补全了需求。测试覆盖不清楚，人知道“这地方我没把握”，AI 觉得“测试过了”就等于“行为安全”。PR 太大，人审不动就变成随便看看。没有 required check，“最好跑一下测试”跟“不用跑”是一回事。

把经验变成门禁，把隐性判断写成 AI 也必须过的检查。不做到这一步，AI 的高产不是资产，是风险。

1. 思维跃迁：小项目 AI 帮你写，大项目 AI 帮你填

小项目里，需求给到 AI，它能产出完整实现——整个项目都装得进上下文窗口，它知道每个函数在系统里的位置。

项目大到一定规模，这件事就不成立了。AI 写得出一个局部，看不到全局。你把“设计一个新子系统”整个交给它，它会产出一套看起来漂亮、但跟现有模块职责重叠甚至循环依赖的设计。不是它不行，是它没有完整上下文。

还有一层：大项目的架构很少是一次设计好的，更多是演进出来的——需求来一波、系统长一块。每一步该往哪走、要不要引入新边界，靠的是人对业务和历史的判断。这种演进式设计里，定方向天然是人活。AI 负责在选定方向里把实现填满。

所以在项目里，人的位置从"实现者"变成了"给 AI 定边界的人"。架构边界、模块职责、接口契约——这些由人定。AI 在格子里填实现、补测试、做机械重构。

定边界比填实现更难，也更值钱。 把精力从"填代码"挪到"定边界、定标准、定门禁"，是 AI coding 真正带来的分工变化。

1.1 上下文工程：不是塞得越多越好

大项目里 AI 最常见的翻车，是改了一处、弄坏它没看见的另一处。根子在一对矛盾上：AI 的上下文窗口有限，代码库近似无限。

上下文工程不是把全仓库塞进去。是每次只把当前任务真正需要的信息放进去。常用手段三个：

- **按需检索**：围绕当前入口、调用方、测试、issue 查相关文件。
- **分层摘要**：先读模块摘要和 public API，需要时再下钻到具体实现。
- **活文档**：让 spec 跟代码一起更新，下一次 AI 和新人都先读它。

塞太多，关键信息被淹没，成本飙升；塞太少，AI 靠猜。功夫全在"太多"和"太少"之间——每次只放对的那部分。

1.2 AI 当活文档用

老系统最贵的不是代码，是"这段代码为什么这么写"的知识。这些知识经常不在文档里——在离职同事脑子里、历史 issue 里、线上事故的复盘里。

AI 在这里的第一个高价值用法，不是改代码。是通读老代码，产出一份"它现在到底在干什么"的 spec。这份 spec 有两层用处：对人，新人接手时先读它，知道入口在哪、状态怎么存的、历史坑在哪；对 AI，下一次改这块时它不再从零读 3000 行，先读这份状态账本。

这接到 §7"让 spec 活起来"。一次性的 spec 很快会腐烂。持续维护的 spec 才是大项目里的知识资产。

2. 让 AI 读懂遗留代码

输入代码：code/01_read_legacy_structure/god_file_order_system/order_system.py。

这是一个写得很糟的订单系统。但它能跑，而且真实——3000 多行塞在一个文件里——`OrderSystem` 一个类包了计价、折扣、VIP、券、税、运费、积分、库存、风控、持久化、通知、报表。核心入口是 250 行左右的 `checkout()`。

代码里没有"这里有 bug"的注释，问题要靠 AI 通读发现。第一步不是让它改代码——是让它只读，先把结构理清楚。

让 AI 输出四件东西：

- 1. public API 和入口清单。
- 2. 职责板块：计价、折扣、税、运费、库存、风控分别散落在哪。
- 3. 状态地图：哪些落 sqlite、哪些是全局配置、库存和事件和审计记录怎么读写的。
- 4. 可疑行为清单：看着像 bug，但重构前必须先原样锁住。

在大项目里，第一份交付物经常不是代码，是结构理解。

```
# 运行遗留系统，确认它可以正常执行
# !cd code && python
01_read_legacy_structure/god_file_order_system/order_system.py
```

2.1 AI 应该找出哪些现状

这份订单系统里，至少有几类行为要让 AI 找出来。不一定是 bug，但重构前必须先锁住：

现状	表现	为什么不能顺手改
VIP 和 percent 券叠加	VIP 折后再打券折	可能已有用户或财务口径依赖
券阈值算子不一致	fixed 用 <code>>=</code> ，percent 用 <code>></code>	边界金额会出现不同结果
免运费基数不一致	报价按券前，下单按券后	同一单可能报价免邮、下单收运费
EU 双重取整漂移	逐行 round + 总额 round	偶尔差 1 分，财务最怕这种
风控拒单库存泄漏	拒单时库存预留没释放	线上库存越卖越少
EU 电子双重折扣	历史折扣分支没删	看起来合理地"清理"会改变金额

更麻烦的是，系统里有几套真相。checkout、calc_v0、calc_v1、calc_v2、RegionalSettlement、checkout_v3、PriceQuoteBuilder——七条结算路径，几乎没有两条算出一样的数。先承认系统里有几套真相，才谈重构。

2.2 带走：架构梳理行动卡

配 [架构梳理行动卡](#)。一套操作规程：

- 输入：一个入口函数、模块或目录。
- AI 动作：只读、列入口、拆职责、画状态、找可疑行为。
- 人审动作：确认 public API、隐式状态、下游调用面有没有漏。
- 完成标准：能说清哪些行为必须先保持不变。

可以直接套用的提示词：

请只读这段遗留代码，不要修改任何文件。

请输出：public API / 入口清单、职责板块、状态地图、多套实现、可疑行为清单，以及第一批应该补的特征测试。

要求：先描述现状，不要给重构方案，不要顺手修 bug。

3. 把现状写成 spec，并生成特征测试

架构梳理完了，下一步不是重构。是锁行为。

特征测试（characterization test，又叫 golden master test）验证的不是“对不对”，而是“改完之后和改之前一不一样”。普通测试面向理想行为，特征测试面向当前行为。

特征测试要锁住现在的行为，哪怕现在的行为看起来是错的。

 **铁律：**没有特征测试，就不许 AI 重构这块。没有安全网让 AI 去改一坨没人看得懂的老代码，是在赌运气。

比如 VIP 和优惠券叠加。看起来可能不合理——但如果重构时顺手改掉，线上金额变了，没人知道是重构改坏的还是修 bug 改坏的。正确做法：先把“会叠加”这个现状锁进测试。要不要改，单独开修 bug 的 PR。

3.0 重构之前先回答三个问题

让 AI 动老系统之前，别上来就问"怎么拆"。先问三个更基础的问题。

为什么重构？ 是为了降低维护成本、隔离风险、改善性能，还是为新功能铺路？目标不清楚，重构很容易变成"把代码整理得更顺眼"。AI 特别爱做这种表面整理。

哪些行为必须不变？ 重构的定义是改变结构、不改变行为。行为不只是返回值——异常、日志、事件、数据库副作用、warning、public import 路径、兼容入口、性能上限、权限边界，都在"行为"的范围内。

怎么回退？ 每一步都要能退回上一个可用状态。小步 commit、小步 PR、特征测试全绿，都是为了让回退有落点。

三个问题回答不清楚，AI 改得越多，坑埋得越深。

3.0.1 行为面清单：不要只盯返回值

特征测试最容易写薄的地方，是只测输入输出。真实系统里，行为面宽得多：

行为面	例子	为什么要锁
返回结构	字段名、嵌套结构、金额精度	下游按字段取值
异常 / 拒绝路径	风控拒单、参数校验失败	失败路径也可能被依赖
数据库副作用	订单、库存、审计记录	不在返回值里，但影响系统状态
事件 / 日志	事件名、日志关键字段	监控和运营可能依赖
warning	deprecation warning 的类型和位置	兼容迁移靠它提醒
public import 路径	包导出、老入口别名	下游代码可能直接 import
性能边界	不能把 $O(n)$ 改成 $O(n^2)$	测试绿也可能线上慢死

回到这个订单系统，sqlite、库存预留、审计记录、老结算入口都在行为面的范围里。

3.1 为什么 AI 让"不敢动"变成了"敢动"

特征测试不是新东西。Michael Feathers 写《修改代码的艺术》是 2004 年。过去它有一个要命的前提：人得先读懂老代码，才写得出锁现状的测试。

对一段没人敢碰、没人能解释的遗留代码，光读懂这一步就可能要几天。很多老代码不是没人想重构——是补测试的成本太高，高到不划算。

AI 改变的就是这一步。它可以快速通读一大段老代码，先列出输入输出和隐式状态，再根据当前行为生成第一版特征测试。人从"从零读完再写测试"变成"审 AI 的结构梳理和测试"。

所以 AI 给重构带来的真变化不只是"更快"——是过去不敢动的地方，现在有机会敢动了。

3.2 锁住现状的四步

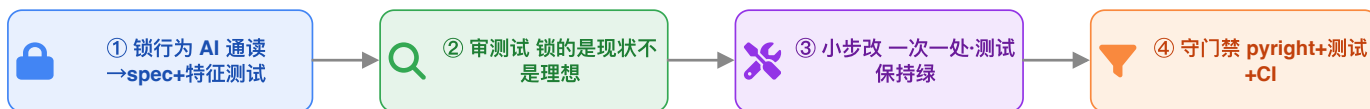
- 1. 让 AI 基于架构梳理写当前行为 spec。
- 2. 让 AI 生成特征测试。
- 3. 人审：测试锁的是现状还是理想？可疑行为有没有原样锁进去？
- 4. 跑绿。

人审的时候重点看这几件事：有没有覆盖可疑行为、sqlite 有没有用临时库隔离、全局状态有没有清理、public API 和返回结构和事件和审计记录有没有覆盖、测试命名是写"锁现状"还是在偷判"应该这样"。

3.2.1 特征测试 vs 普通测试

维度	普通测试	特征测试
问题	这个行为对不对？	改完和改前一不一样？
前提	你知道正确答案	你只知道当前系统正在这么跑
适合	新功能、新规则	遗留代码、安全重构
现状有 bug	写正确行为	先锁住 bug 的现状，bug 单独修
失败含义	实现不符合预期	行为发生了变化

不然很容易顺手让 AI 把看着明显的 bug 一起改掉——重构和修 bug 混进同一步，出了问题分不清是哪边引入的。



两条铁律：没有特征测试不许 AI 重构这块 · 一个 PR 只做一件事（重构与改 bug 分开）

```
# 运行特征测试，确认 6 处可疑现状均被锁住
# !cd code && pytest 01_read_legacy_structure/ -q
```

3.3 带走：特征测试行动卡

配 [特征测试行动卡](#)。核心就几条：

- 特征测试锁现状，不锁理想。
- 现状像 bug 也先锁住。
- bug 单独 PR。
- 没有特征测试，不许 AI 重构这块。

可以直接套用的提示词：

基于刚才的架构梳理，请先写当前行为 spec，然后生成特征测试。

要求：测试锁住当前行为，不判断当前行为是否合理；可疑行为必须原样锁住；数据库、全局变量、缓存、配置必须 reset 并隔离；生成后说明每条测试锁住了哪条现状。

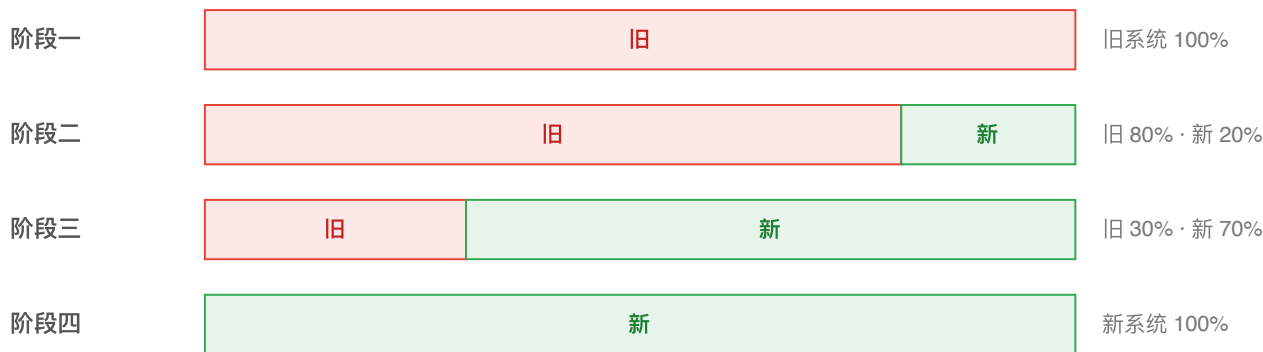
不要修改业务代码。

4. 🔧 小步重构，并用红绿测试验证行为没变

测试全绿了，才允许 AI 重构。

重构的定义是：改变结构，不改变行为。AI 最容易把“顺手优化”“顺手修 bug”“顺手改命名”混在一起，这条线必须划得特别清楚。

绞杀者模式：新实现在旧系统外围一点点长出，逐步接管，全程不停机



每一步都带对照测试、可切回旧路径；旧的被逐步“绞杀”下线

顺序是这样的：

1. 保留 `checkout()` 入口。
2. 让 AI 一次只抽一个职责块，比如 `_price_lines`、`_apply_vip`、`_apply_coupon`、`_tax`、`_shipping`。
3. 每抽一块跑一次特征测试。
4. 故意把"VIP 和券叠加"改成"只取更优的一个"。
5. 对应的特征测试变红。
6. 解释这个红灯怎么处理：重构破坏行为就撤回；如果确认是 bug，另开 PR 修。

这一步最该看到的就是红灯亮起来——它不是失败，是安全网兜住了。

4.1 怎么把大改造拆成安全小步

特征测试兜了底，还得控制重构的步子。

切分 + 接口对齐（有的资料叫"暴力拆解 + 精准缝合"）：先按职责把大函数粗粒度切开，再逐个把切口的接口对齐。AI 擅长抽函数、搬代码、补样板，人负责判断切在哪。

保留入口，内部委托：对外入口不动，内部逐步迁移到新函数或新模块。这就是绞杀者模式（Strangler Fig，有的资料写作"桑树模式"）缩小到一个函数上的版本——旧入口还在，下游无感，新结构一点点长出来。

一次一个边界：一次只动一个函数、一个职责块、一个文件。diff 小，review 才有效。diff 大，测试红了也不知道是哪一步引入的。

红灯先停：测试红的时候，不让 AI 继续扩大修改范围。先判断是测试写错了、状态没隔离，还是行为真的变了。

4.2 带走：安全重构执行卡

配 [安全重构执行卡](#)。执行纪律：

- 一次只动一个边界。
- 入口不变，内部委托。
- 每步跑测试。
- 重构和修 bug 分开。
- 一个 PR 只做一件事。

可以直接套用的提示词：

现在特征测试已经全绿。请做一次小步安全重构。

约束：保留 public API / 入口 / 返回结构；一次只重构一个职责块；每完成一步运行全部特征测试；测试失败时先解释哪条行为变化，再给最小修复；不要顺手修 bug，不要新增功能。

请先给本步计划，等我确认后再改代码。

5. 另一种难：问题藏在结构里

输入代码：[code/01_read_legacy_structure/advanced_modular_coupling/](#)。

前面那个上帝文件难在体量——3000 行、一个函数 250 行、几套真相并存。但至少每一处可疑行为还能在单个文件里读出来。

真实大项目还有另一种难。代码看起来已经拆成模块了，但模块之间靠全局变量、近循环依赖、import 副作用和共享 context 绑在一起。单看每个文件都正常，合起来才出事。这种代码只需要让 AI 画出依赖图和状态图，不必展开完整重构。

几个真实的结构坑：`engine` import `pricing`，`pricing` 又回读 `engine.REGION`——要看懂小计怎么算，得先知道 `engine` 当前设的区域。多个步骤读写同一个 `CTX` 字典，数据靠时序传而不是靠参数传。import 包的时候偷偷注册促销券，测试之间会串台。订单小计和积分口径分属不同模块，改了一处另一处不会跟着变。public import 路径也是行为，重构时不能随便挪。

结论：结构复杂的时候，先画依赖图和状态图。这张图画出来之前，spec 和特征测试没有意义。

5.1 带走：结构梳理行动卡

配 [结构梳理行动卡](#)。和 §2 的架构梳理不同——§2 面对的是单文件上帝类，主要拆职责；这里面对的是多文件耦合，核心输出是依赖图和状态地图。

可以直接套用的提示词：

请只读这个目录下的代码，不要修改任何文件。

先画两张图：

1. 依赖图：每个文件 `import` 了哪些其他文件，特别标出近循环依赖。
2. 状态地图：哪些是全局/持久化状态（数据库、模块级变量、配置文件、全局字典），哪些步骤在读写它们。

再列出 `public API`、`public import` 路径、下游可能绕过的内部调用点、事件/日志/审计的格式和触发位置。

最后说明：如果重构时只改其中一个文件，哪些地方最容易出问题。不要给重构方案，先画图。

```
# 模块化泥潭同样是真实可跑代码
# !cd code/01_read_legacy_structure/advanced_modular_coupling && python runner.py
# !cd code/01_read_legacy_structure/advanced_modular_coupling && pytest -q
```

6. Git 管理如何保障质量

进入 GitHub workflow 之前，先看 Git 本身。

Git 不只是版本历史——在 AI coding 里，它是质量控制轨道：把 AI 的高产出切成可理解、可追踪、可回退的小块。

Git 动作	质量价值	对 AI 产出的意义
issue	定义目标、验收标准、不要动的范围	给 AI 划边界
branch	隔离风险	半成品不污染主干
commit	固化一步变化	出问题能定位是哪一步

Git 动作	质量价值	对 AI 产出的意义
diff	让变化可审查	人审看代码差异，不看聊天记录
blame / bisect	追溯问题来源	线上问题能倒查引入点
PR	把变更变成团队审查对象	评论、讨论、回退都有载体

"一个 PR 只做一件事"不是流程洁癖，是质量保障。重构、修 bug、新功能混在一起，出了问题没人知道该回滚哪一部分。前面安全重构一直强调小步，原因就在这里——小步测试、小步 commit、小步 PR，人审和机器验证才使得上劲。

6.1 几个 Git 动作在质量控制里的实际作用

diff：让人审有焦点。没有 diff，评审只能靠听 AI 描述。有 diff，人可以逐行看哪些代码真的变了。

commit：把变化切成时间片。小 commit 不是为了历史好看，是为了出问题能退回一小步。

revert：让回滚变成工程动作，而不是手动"凭记忆改回去"。AI 改坏时，能 revert 一个 PR，比在聊天里要求它"恢复原样"可靠得多。

bisect：测试突然红了，没人知道哪次改动引入的。git bisect 二分定位坏提交。AI 产出越多，这个能力越重要。

blame：不是为了追责，是为了找上下文。看到一行代码是谁、因为什么 issue 改的，才能判断它是历史包袱还是业务约束。

cherry-pick：一个大分支里只有某个小修复是安全的，可以把安全部分摘出来，而不是整包合并。

这些能力一起把 AI 的高产出压缩成可管理的工程单元。

6.2 Git 和特征测试怎么配合

安全重构的实际节奏：

1. 开分支：refactor/<issue>-split-checkout。

2. 第一组 commit 只加 spec 和特征测试，不改业务代码。
3. 第二组 commit 每次只抽一个职责块。
4. 每个 commit 后跑特征测试。
5. 测试红，优先 revert 最近一步，而不是让 AI 继续大改。
6. PR 里说明：行为不变，锁住了哪些现状，没修哪些疑似 bug。

人审看到的链路很清晰：先补安全网，再小步重构，再用测试证明行为没变。

7. 让 spec 活起来：从一次性快照到状态账本

前面生成的 spec 和特征测试，如果只停在本次任务里，很快会变成过期文档。

这套结构在 code/04_live_spec_template/：

- **AGENTS.md**：把项目规则写给 agent。
- **spec/governance/**：长期生效的规则。
- **spec/planned/**：已设计、未落地。
- **spec/implemented/**：已落地，带实现锚点。
- **spec/archived/**：废弃或搁置。
- **SPEC_TEMPLATE.md**：单条 spec 的模板。

核心循环：issue 进来先分类。局部 bug 且期望已定义，最小修复 + 回归测试 + 同步文档。影响功能、架构、公共接口、兼容线的，先 spec 后实现。PR 合并时，实现、测试、文档、spec 状态必须一致。

把这套建立起来，spec 才从一次性快照变成状态账本。

7.0 为什么光有 README 不够

很多项目不是没有文档，是文档和代码脱节。脱节的原因通常不是大家不认真——是文档没有状态、没有归属、没有验收动作。

活 spec 要解决三个问题：这条 spec 现在是计划中、已实现、长期规则还是已经废弃？它对应哪些代码、测试、issue、PR？代码改了之后，spec 有没有跟着移动状态？

没有这三件事，spec 只是“写得比较正式的聊天记录”。配上这三件事，它才是 agent 和团队共同维护的状态账本。

7.1 issue 分类是 spec 治理的入口

不是每个 issue 都要先写一份长 spec。关键是分类：

issue 类型	处理方式
局部 bug，期望已经清楚	最小修复 + 回归测试 + 同步文档
涉及新功能	先写 planned spec，再实现
涉及公共接口	先对齐兼容策略，再动代码
涉及架构边界	先画方案，别让 AI 自己移动目录
多个 issue 像同一根因	先抽象统一方案，别逐个打补丁
分不清影响面	按"影响设计"处理，先停下来问

AI 最容易把每个 issue 都当成局部补丁。大项目里，很多局部 bug 其实是同一个设计缺口的不同症状。

7.2 带走：spec ↔ issue 治理行动卡

配 spec ↔ issue 治理行动卡。一条核心铁律：

一个改动不算完，直到实现、测试、文档、示例、兼容信息、spec 状态讲的是同一个故事。

可以直接套用的提示词：

请基于当前项目建立一个最小 spec 状态账本：生成 AGENTS.md，建立 spec/README.md，按 governance/planned/implemented/archived 分类，把当前行为 spec 整理成单条 spec，并给现有 issue 分类。

不要写业务实现，先建立治理结构。

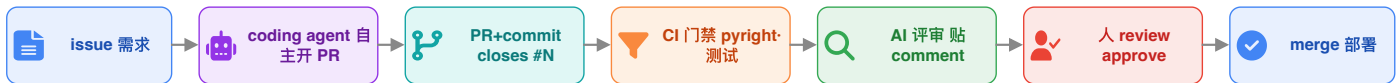
8. 把 AI 产出放进 GitHub workflow

输入代码：[code/05_github_pipeline/](#)。

这里不讲发包，不讲发布链路。只讲最小可用的团队质量管线：

issue -> branch -> commit -> PR -> GitHub workflow(ruff + pytest) -> 人审 -> merge

一条 GitHub 原生的 AI 协作回路：issue 进来，AI 当执行者，人是最后的决策者



CI 门禁与 AI 评审都是辅助，merge 由人按确认 —— coding agent 再自主，终点也是一个必须人审批的 PR (AI 无豁免权)

为什么只用 lint + pytest？因为重点是 workflow 的门禁能力，不是 Python 包发布。lint 挡住明显代码质量问题，pytest 挡住行为回归。两者放一起足够说明一件事：AI 产出进主干之前，必须先过确定性检查。

8.1 workflow 强在哪："自动 + 强制"

GitHub workflow 的价值不在 YAML 本身。在于它把"建议你跑一下"变成了"仓库自动执行，不通过不能合并"。

- **自动触发**：push、pull_request、手动 dispatch、定时任务、路径过滤都可以触发。
- **环境一致**：统一 Python 版本和依赖安装方式，消灭"我本地可以"。
- **结果可见**：每个 PR 都能看到哪一步红，红在哪条命令。
- **可设门禁**：配合 Branch protection，把 CI 设为 required check，红了不能 merge。
- **可扩展**：今天是 lint + pytest，明天可以加安全扫描、覆盖率、文档构建、矩阵测试。

打开 code/05_github_pipeline/.github/workflows/ci.yml，看这条最小 workflow：

```
on:
  push:
    branches: [main]
  pull_request:

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -e ".[dev]"
      - run: ruff check src tests
      - run: pytest -q
```

就这几行。足够把 AI 产出和人产出放进同一道门。

8.1.1 workflow 可以逐步升级，不要一上来就做复杂

这里只演示 lint + pytest，因为它是最小可用门禁。真实项目可以逐步加，但每一步都要有明确目的：

能力	什么时候加
lint	一开始就加，挡住低级质量问题
pytest	一开始就加，挡住行为回归
path filters	大仓库里，只在相关目录变更时跑对应 job
matrix	多 Python / Node / OS 版本兼容时加
coverage	团队开始关心测试缺口时加
security scan	有依赖风险、供应链风险时加
scheduled workflow	定期跑慢测试、全量检查、依赖巡检
workflow_dispatch	需要人工触发专项检查时加

重点不是一次把 YAML 写满。是把"重要质量判断"逐步从人工记忆搬进自动门禁。

```
# 本地验证: lint + pytest 全绿即可
# !cd code/05_github_pipeline && pip install -e ".[dev]" && ruff check src
tests && pytest -q
```

8.2 从变更链路看这条管线

从变更链路看，这条管线是这样走的：

1. issue 写清目标、验收标准、不要动的范围。
2. agent 或人开分支。
3. 小步 commit，commit 写 **Refs #N**。
4. 开 PR，PR 写 **Closes #N**。
5. workflow 自动跑 lint + pytest。
6. required check 全绿后，人 review。

7. 人决定 merge。

workflow 是机器门禁，人审是语义门禁。lint 和测试能判断格式、静态规则、行为回归——但不能判断架构方向对不对、业务语义值不值得改、风险能不能接受。

8.2.1 AI review 可以用，但不能当门禁

PR 上可以挂 AI 辅助评审，让它先看边界、风格、潜在 bug、遗漏测试。但要说清楚：AI review 是辅助，不是门禁。

原因很简单。AI review 可能漏。AI review 的标准会随模型版本漂移。AI review 无法替团队承担业务风险。AI review 不能替代 lint / pytest 这种可复现检查。

主干门禁永远应该是：**确定性 workflow + required review**。AI review 可以提高评审效率，但不能决定能不能合并。

8.3 带走：GitHub AI 管线行动卡

配 [GitHub AI 管线行动卡](#)。核心是把 AI 的产出变成工程对象：

- issue 划边界。
- branch 隔离风险。
- commit 定位责任。
- PR 形成审查对象。
- workflow 强制 lint + pytest。
- 人审决定 merge。

可以直接套用的提示词：

请按这个 GitHub issue 工作：先复述目标、验收标准和不要动的范围；给出最小计划；创建只解决本 issue 的小步修改；提交信息包含 Refs #<issue号>，PR 正文包含 Closes #<issue号>；运行 workflow 的本地等价命令 `ruff check src tests && pytest -q`；如果失败，只根据失败信息做最小修复。

不要自动合并，不要扩大 scope。

9. 收束：生成—验证—修正质量飞轮

回头看，前面每一步其实是同一条飞轮在转：

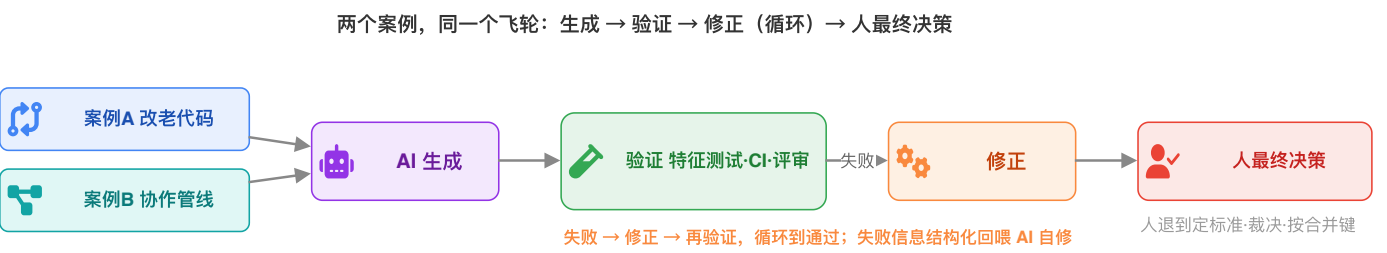
AI 生成

-> spec / 特征测试 / lint / pytest / PR 模板验证

-> 失败信息结构化回喂给 AI

-> AI 修正

-> 人做最终裁决



每一步的对应关系：读懂遗留结构是给 AI 正确上下文。特征测试验证行为变没变。小步重构控制 diff 和回退面。Git 让变更可追溯、可审查、可回退。活 spec 让知识持续对账。GitHub workflow 把验证变成团队门禁。

飞轮里有一个常被误解的环节：让 AI 给自己的产出补边界用例，再用这些用例反过来测自己。这不是机器学习里的对抗样本攻击——这是"生成对照、边界用例自测"。两回事。而且最好让出题的 AI 和被测的产出分开，否则它对着自己的理解出题，测不出理解本身的错。

大项目里 AI 的可信度，不来自它说"我完成了"。来自你把它放进了 spec、测试、Git、workflow、PR、人审 gate 组成的系统里。

9.0 AI 比人更需要制度

对人来说，制度是外部约束。对 AI 来说，制度几乎是可信度的全部来源。

一个靠谱工程师身上有很多隐性刹车：线上事故的压力、代码审美、对历史坑的敬畏、觉得不对劲时会停下来问。AI 没有这些。它不会因为 PR 有 bug 被追责，也不会因为连续犯错自动长记性。

所以对 AI，规则不能只写在聊天里。必须写进 issue 模板、PR 模板、AGENTS.md、spec 状态账本、lint 和 pytest、Branch protection、required review。

这不是流程主义。这是让 AI 能被团队安全使用的基本条件。

9.1 人退到哪里

人的位置变了：

- 人定边界。
- 人定验证标准。
- 人审 AI 生成的 spec 和测试。
- 人裁决机器判断不了的业务语义。
- 人按 merge 键。

AI 做大量体力活：读代码、列现状、补测试、抽函数、修 lint、按失败日志改。人的价值变成定义标准和最终裁决。

这套飞轮怎么从某几个人的习惯变成整个团队的共同纪律——从"超级个体"到"超级团队"——下一篇专门讲。

总结

大项目里 AI 的价值不在写得更快。在三件过去很难、现在有机会做的事：

- 驾驭一个人读不完的代码库：靠架构梳理、上下文工程、活文档。
- 敢动一段没人敢碰的老代码：靠特征测试先锁住现状。
- 让 AI 产出进入团队秩序：靠 Git 的可追溯性和 GitHub workflow 的确定性门禁。

从头到尾用的不是某个工具，是同一条纪律：

先理解，再锁住；先小步，再验证；先入 Git，再进主干；先过 workflow，再由人裁决。

 **课后：**回到自己的项目，挑一个最不敢动的老入口。先别重构——先用架构梳理行动卡让 AI 只读，再用特征测试行动卡锁住 3 条关键现状，最后把这份 spec 放进状态账本。这三步做完，再谈让 AI 改。

常见翻车处理

AI 架构梳理漏了关键入口：不要继续生成测试，先追问 public API、调用方、import 面。

特征测试不稳定：先查数据库、全局状态、时间、随机数、缓存。测试不稳定，不许重构。

AI 把疑似 bug 顺手修了：撤回，先锁现状。如果要修，另开 issue 和 PR。

重构 diff 太大：要求拆小，按单个职责块重新来。大 diff 让 review 失效。

workflow 红但 AI 解释说没问题：以 workflow 为准。让它根据日志做最小修复，不要扩大 scope。

PR 里混了重构、bug、新功能：拆 PR。混合 PR 是大项目质量事故的温床。

spec 写完没人维护：把 spec 状态迁移放进 PR checklist，不更新 spec 就不算完成。